

The Most Efficient Protocol for PAKE: What Exact Stuff Do You Need to Hash at the End?

Jiayu Xu

Oregon State University, xujiay@oregonstate.edu

1 Introduction

2 Security Analysis of the “Hash Everything” Version

2.1 The Simulator

2.1.1 Simulation Strategy

A large portion of the simulator is routine; below we explain the idea behind the non-trivial parts.

Simulating honest P -to- P' message. When P 's session begins, $\mathcal{S}$ must simulate its message (s, T) to P' . This can be easily done: just sample (s, T) at random. However, every time $\mathcal{A}$ queries $H_2(\text{pw}', T)$ (let the result be t'), $\mathcal{S}$ needs to generate a KEM key pair (pk', sk') for later use (see the “Extracting password guess from malicious P' ” paragraph below) and make sure that (pk', sk') is consistent with the RO queries. This can be done as follows: $\mathcal{S}$ computes

$$r' := s \oplus t', \quad R' := T/pk',$$

and programs $H_1(\text{pw}', r') := R'$. This also allows $\mathcal{S}$ to store the association between pw' and (pk', sk') . (Note that (pk', sk') needs to be freshly generated every time $\mathcal{A}$ queries $H_2(\star, T)$; otherwise R' never changes and $\mathcal{A}$ can easily find collisions in H_1 .)

Extracting password guess from malicious P . When the adversary $\mathcal{A}$ sends a message (s^*, T^*) to P' , if it is different from the honest message (s, T) , the simulator $\mathcal{S}$ must extract a password guess (if any) and send a corresponding TestPwd command to $\mathcal{F}_{\text{PAKE}}$. There are two types of attack, which need to be addressed separately:

1. “Normal” online attack: $\mathcal{A}$ executes P ’s algorithm on a password guess pw^* . That is, $\mathcal{A}$ samples public key pk^* and randomness r^* , computes

$$\begin{aligned} R^* &:= H_1(\text{pw}^*, r^*), & T^* &:= pk^* \cdot R^*, \\ t^* &:= H_2(\text{pw}^*, T^*), & s^* &:= t^* \oplus r^*, \end{aligned}$$

and sends (s^*, T^*) to P' .

It takes a while to realize how to extract pw^* from (s^*, T^*) . First, $\mathcal{S}$ should scan all $H_2(\star, T^*)$ queries, which yields multiple pw^* values (and multiple t^* values); $\mathcal{S}$ must decide which one is the “actual” password guess. This can be done via the following. For each such (pw^*, t^*) pair, $\mathcal{S}$ computes the corresponding $r^* := s^* \oplus t^*$, and checks if (and when) $H_1(\text{pw}^*, r^*)$ was queried. *With overwhelming probability, there is at most one such H_1 query made before the $H_2(\text{pw}^*, T^*)$ query*, from which $\mathcal{S}$ can extract the “actual” password guess pw^* . (This special $H_1(\text{pw}^*, r^*)$ query is called the “anchor query” in [MRR20, JRX25].)

2. *Related key attack*: This attack works only after $\mathcal{A}$ receive an honest (s, T) pair from P . $\mathcal{A}$ skips sampling pk^* or r^* ; instead, $\mathcal{A}$ picks any $\Delta \in \mathbb{G}$, computes $T^* := T \cdot \Delta$ and

$$\begin{aligned} t^* &:= H_2(\text{pw}^*, T), & (t^*)' &:= H_2(\text{pw}^*, T^*), \\ \delta &:= t^* \oplus (t^*)', & s^* &:= s \oplus \delta, \end{aligned}$$

and sends (s^*, T^*) to P' .

Let us first explain what $\mathcal{A}$ is trying to achieve here. The same randomness r yields two valid messages, (s, T) and (s^*, T^*) , such that

$$s \oplus s^* = H_2(\text{pw}, T) \oplus H_2(\text{pw}, T^*),$$

which allows $\mathcal{A}$ to pick T^* and “reverse engineer” the corresponding s^* if it guesses pw correctly. (s^*, T^*) implicitly encrypts $pk^* = T^* / H_1(\text{pw}, r)$, which $\mathcal{A}$ can compute as $pk \cdot (T^* / T)$.

To extract pw^* from (s^*, T^*) , $\mathcal{S}$ should scan all $H_2(\star, T)$ and $H_2(\star, T^*)$ queries (which yield multiple candidate pw^* values) and check which one satisfies

$$H_2(\text{pw}^*, T) \oplus H_2(\text{pw}^*, T^*) = s \oplus s^*.$$

With overwhelming probability, at most one pw^* will satisfy this equation.

Extracting password guess from malicious P' . When the adversary $\mathcal{A}$ sends a message (c^*, τ^*) to P , if $\mathcal{A}$ is not passive (i.e., it is not the case that $(s^*, T^*) = (s, T)$ and $(c^*, \tau^*) = (c, \tau)$), the simulator $\mathcal{S}$ must extract a password guess (if any) and send a corresponding `TestPw` command to $\mathcal{F}_{\text{PAKE}}$.

$\mathcal{S}$ should find the H_{tag} query whose result is τ^* , which includes (pw^*, pk^*, K^*) . However, pw^* constitutes a password guess only if it is consistent with pk^* and K^* , i.e., $K^* = \text{Decap}(sk^*, c^*)$ for sk^* corresponding to pk^* . $\mathcal{S}$ can check this

condition because it has stored the correspondence between $\mathbf{pw}^*$ and (pk^*, sk^*) (see the “Simulating honest P -to- P' message” paragraph above), and can use sk^* to check if the equation holds. (This extraction strategy does not work anymore if $\mathbf{pw}^*$ is not included in H_{tag} . In later sections we will explain how to simulate the “not hashing $\mathbf{pw}$ ” version of the protocol.)

References

- [JRX25] Jake Januzelli, Lawrence Roy, and Jiayu Xu. Under what conditions is encrypted key exchange actually secure? In *EUROCRYPT 2025*, pages 451–481, 2025.
- [MRR20] Ian McQuoid, Mike Rosulek, and Lawrence Roy. Minimal symmetric PAKE and 1-out-of-N OT from programmable-once public functions. In *CCS 2020*, pages 425–442, 2020.